Vigil MDR Implementation Guide

How the Vigil MDR lab project is built, how it runs, and what remains. Version of 31 October 2026.

1. Overview

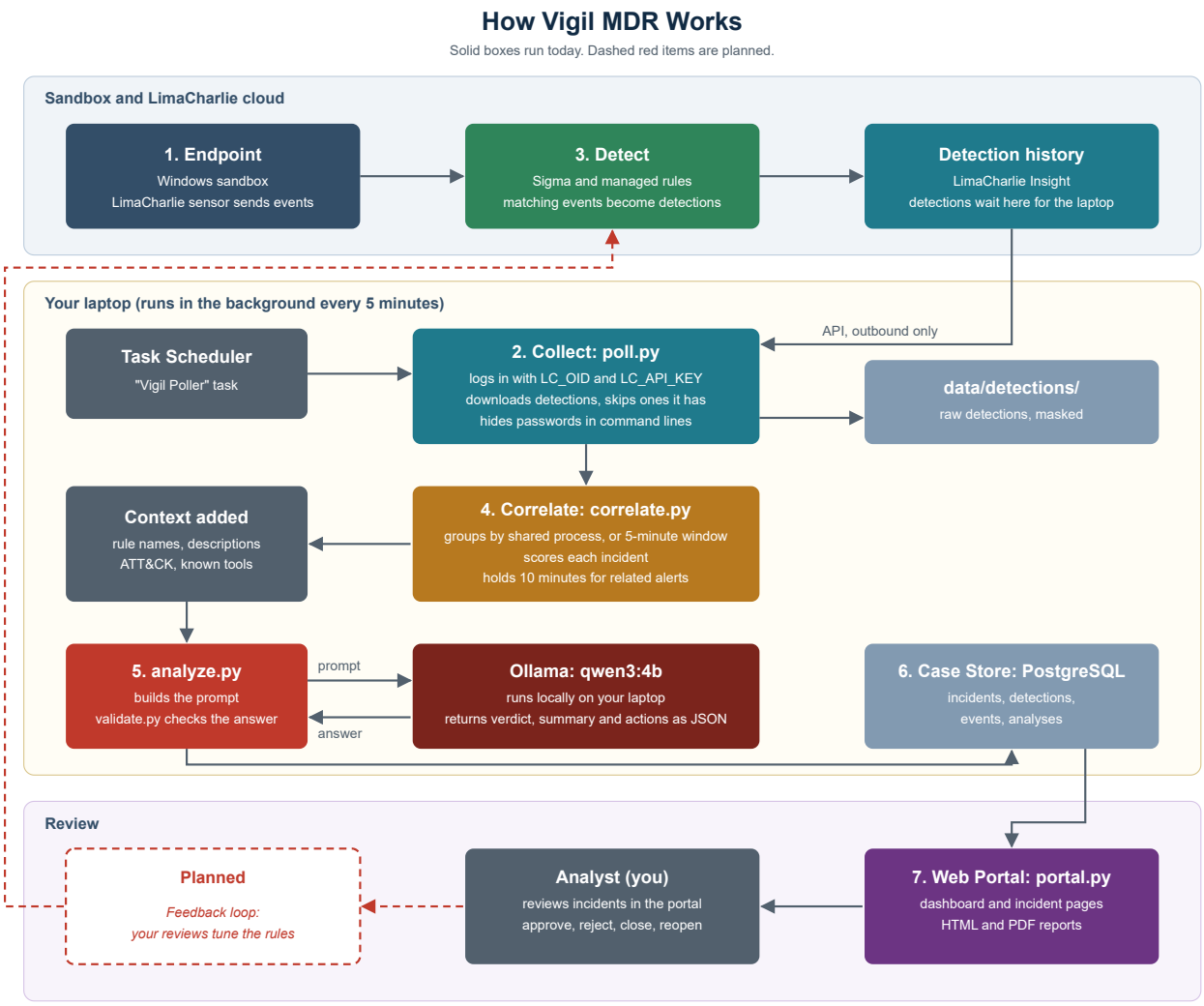
Vigil is a personal Managed Detection and Response lab. It collects detections from LimaCharlie, groups related alerts into scored incidents, asks a local AI model for a triage opinion, stores the results in PostgreSQL, and gives an analyst a web portal to review them.

It's a learning project, not a production security service. It doesn't block attacks, and a person makes the final decision on every incident.

> **Current limitation:** AI verdicts aren't reliable yet and accuracy hasn't been measured. In testing, two nearly identical incidents received opposite verdicts. Section 8 has the details.

New to this project? Sections 2 to 11 explain the design. Section 12 walks through every build step in order, with the commands, what they do, and how to check them. Section 13 explains the technical terms.

2. Architecture



Stage	Component	Status
1. Endpoints	Windows sandbox with the LimaCharlie sensor	Working
2. Collect	`poll.py`, run every 5 minutes	Working
3. Detect	LimaCharlie Sigma and managed rules	Working, needs tuning
4. Correlate	`correlate.py`	Working, needs tuning
5. AI Analyze	`analyze.py`, `build_prompt.py`, Ollama	Running, accuracy not measured
6. Case Store	PostgreSQL 17 through `db.py`	Working
7. Web Portal	`portal.py`, FastAPI, Jinja, HTMX	Working
Analyst review	Approve, reject, close and reopen	Working
Feedback loop	Reviews tune detection rules	Planned

Key design decisions

- **Polling replaced the webhook.** The laptop makes outbound requests only, and detections missed while it's off are collected from LimaCharlie's history on the next run.

- **The AI runs locally.** Detection data is sent to Ollama on the laptop, not to an external AI service.
- **Code owns the facts.** Grouping, scoring, ATT&CK techniques and timelines come from code. The model only provides an opinion.
- **Evidence is untrusted.** Command lines can be written by an attacker, so they stay separate from trusted context and are escaped on every page.

3. Project layout

Path	Purpose
`poll.py`	Main pipeline: fetch, mask, save, correlate, analyze, store
`adapters/limacharlie.py`	Converts a detection into a simple incident shape and masks secrets
`correlate.py`	Groups detections and calculates scores
`analyze.py`	Calls Ollama, retries once, and defangs the answer
`build_prompt.py`	Builds the prompt from core rules and matching playbooks
`validate.py`	Checks the schema and rejects invented event IDs
`defang.py`	Makes IPs, domains, URLs and emails unclickable
`db.py`, `schema.sql`	PostgreSQL storage and table definitions
`portal.py`	Web portal, login and security controls
`process_tree.py`	Builds process trees from LimaCharlie process links
`report.py`	Builds the HTML and PDF reports
`charts.py`	Draws the dashboard charts on the server
`prompts/`	`core.md` and 11 playbooks
`schema/verdict.schema.json`	Required format for AI answers
`templates/`, `static/`	Portal pages, styling, font, logo, HTMX and `app.js`
`tools/`	Daily status and portal password scripts
`data/`	Local detections, logs and reports, excluded from Git

4. Data flow

1. The Windows task "Vigil Poller" runs `poll.py` every 5 minutes. A new run doesn't start while one is active.
2. `poll.py` exchanges `LC_OID` and `LC_API_KEY` for a short-lived token, then requests detections since the last check, plus 10 extra minutes. It follows `next_cursor` through all pages.
3. Passwords and tokens in argument values such as `--auth=` are replaced with `[REDACTED]`, then new detections are saved to `data/detections/`.
4. All saved detections are regrouped into incidents and synced to PostgreSQL.

5. An incident becomes ready 10 minutes after its last detection, so related alerts can join it.
6. Up to 10 incidents per run go to Ollama, highest score first. An incident is analyzed again if its detections change.
7. The verdict is validated, defanged and stored. If Ollama is unreachable, the run stops and a later run retries.

5. Correlation and scoring

Grouping rules

Rule	Value
Scope	Same host
Shared process link	Up to 30 minutes apart
Timing only	Up to 5 minutes apart, not for known tools
Maximum incident span	1 hour
Prompt limits	12 unique events, command lines cut at 400 characters, 6 rules

Process links use LimaCharlie atoms, which are unique IDs. They're more reliable than process IDs, which Windows reuses.

Score

Factor	Effect
Base	Highest rule severity in the incident
Several ATT&CK techniques	+5 per extra technique, up to +15
Several rules	+5 per extra rule, up to +10
LSASS targeted	+20
Known tool involved	-30

Scores stay between 0 and 100. A score measures alert severity, not the likelihood of an attack.

6. AI analysis

The model is `qwen3:4b`, called through Ollama's local API with temperature 0, an 8192-token context and JSON output forced to match the schema.

1. `build_prompt.py` maps the incident's ATT&CK techniques to playbooks and keeps at most 4 for a normal analysis.
2. Trusted data goes in a `<context>` block, and events go in `<evidence>`.
3. `validate.py` checks the schema and every cited event ID in claims and the attack chain.
4. On failure, the errors are sent back for one retry. If it still fails, the incident is marked `failed`.

5. `defang.py` masks indicators in the answer, even if the model missed them.

The answer contains a verdict, confidence, severity, summary, attack chain, claims, indicators, evidence requests and recommended actions.

7. Database

Table	Holds
`incidents`	Host, score, time range, grouping method and context
`detections`	Masked detections, rule, time, ATT&CK techniques and incident link
`events`	Process events used as evidence
`analyses`	Every AI verdict, including older versions
`reviews`	Analyst decisions, reject reasons and summary edits. These have no foreign key, so they survive regrouping

PostgreSQL 17 is installed from conda-forge in `F:\PostgreSQL`, because the official download server refused the connection and drive C: was low on space. It accepts local connections only, and the application uses a dedicated `vigil` login.

8. Testing and known limits

Verified

- The saved detection count matched LimaCharlie's 7-day history, with none missing.
- Signed-out visitors can't open portal pages, and forged cookies, invalid hosts and cross-site form posts are refused.
- Injected script text displays harmlessly in the portal and process tree.
- Unmasked `--auth=` values weren't found in saved detections or reports.

AI test results

Test	Result
First Orborus and LSASS incident	`benign`, high confidence, calling LSASS access normal without proof
Second Orborus and LSASS incident	`true_positive`, critical, with an unsupported claim of credential theft
Atomic Red Team session	`true_positive`, critical, marked failed for citing an invented event ID, with unsupported claims of lateral movement and data theft
Speed	About 3 to 7 minutes per typical incident with the full prompts, and 14 minutes for the largest

Known limits

- The model may be too small to apply the full guidance reliably.
- The laptop is the server, so the site and analysis stop when it's off or asleep.
- The portal and tunnel don't start by themselves after a restart.
- The quick tunnel's public address changes on restart and is meant for testing.
- The portal has a single password and no two-step sign-in.
- The database has no backups yet.
- Automatic masking only covers common argument names.
- Process trees only include processes that appear in detections.

9. Portal security

Control	Implementation
Login	Every page except login and static files needs a password
Password storage	Salted scrypt hash in a Windows user variable
Sessions	Signed cookie, HttpOnly, SameSite Strict, Secure in public mode, 8-hour limit
Guessing	5 failed attempts blocks an address for 15 minutes
Forged requests	Forms must come from the portal's own origin
Hosts	Only local names and `*.trycloudflare.com` are accepted
Page safety	Automatic escaping and a Content Security Policy
Network	Portal listens on 127.0.0.1, with public HTTPS through the tunnel

10. Operations

Item	Value
Scheduled tasks	"Vigil Poller", "Vigil PostgreSQL", "Vigil Daily Status". The portal and tunnel are started by hand
User variables	`LC_OID`, `LC_API_KEY`, `VIGIL_DB_PASSWORD`, `VIGIL_PORTAL_HASH`, `VIGIL_SESSION_SECRET`
Logs	`data\poll.log`, `data\portal.log`, `F:\PostgreSQL\postgres.log`
Reports	`data\reports\`
Setup and daily use	`instructions.md`

> Never commit `data/`, `.venv/`, API keys or passwords. Reports and detections contain hostnames, usernames, IP addresses and command lines.

11. Remaining work

1. Add code rules for known tools touching LSASS, and block contradictory verdicts.
2. Test a larger Ollama model if the laptop can run it.
3. Find out whether Orborus access to LSASS is normal.
4. Build an eval set and measure verdict accuracy.
5. Tune grouping, scoring and noisy detection rules.
6. Set up database backups, and confirm services recover after a restart.
7. Decide how the portal and tunnel should start by themselves.
8. Build the feedback loop from reviews into rule tuning.

The live checklist is in `SPRINT_STATUS.md`.

12. Step-by-step build walkthrough

This section follows the order the project was built in. Each step says what it's for, the commands used, and how to confirm it worked.

> **Before you copy commands:** run them in PowerShell on Windows, unless a step says otherwise. Replace placeholders such as `your-api-key`. This project was built in `F:\Gitrepo\vigil mdr`, so change that path if yours differs. For a fresh install, `docs/Vigil-MDR-Setup-Guide.pdf` is shorter. Run the sandbox commands only on an isolated test machine.

Step 1: Create the test machine and connect LimaCharlie

Why: Vigil needs a safe machine to watch. LimaCharlie's sensor sends that machine's activity to the cloud.

1. Create a Windows virtual machine with NAT networking, and turn off shared folders and clipboard.
2. Take a snapshot, so you can undo any test.
3. Create a LimaCharlie organization and installation key, then install the sensor on the virtual machine.

Run this on the virtual machine, then find it in the sensor's timeline:

```
whoami /all
```

Check: you can see the command and its parent process in LimaCharlie.

Step 2: Create detections

Why: a sensor only records events. Rules decide which events become alerts, called detections.

Enable Insight for detection history, and enable Sigma and managed rule sets. Then run this harmless encoded test on the virtual machine:

```
powershell.exe -nop -enc VwByAGkAdABlAC0ASABvAHMAdAAgAFYAaQBnAGkAbAAgAHQAZQBzAHQA
```

Check: the detection appears on LimaCharlie's Detections page.

Step 3: Create the project folder and Python workspace

Why: a virtual environment keeps this project's Python packages separate from the rest of your laptop.

```
cd "F:\Gitrepo\vigil mdr"
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install requests
mkdir data
```

What these do: go to the folder, create the workspace, turn it on, install a web request tool, and create a data folder.

Check: your prompt starts with `(.venv)`.

Step 4: Connect to LimaCharlie safely

Why: the laptop needs permission to read detections. In LimaCharlie, create an API key named `vigil-poller` with only the `insight.det.get` permission. Never use the temporary Org JWT, which has far wider access.

```
[Environment]::SetEnvironmentVariable("LC_OID", "your-org-id", "User")
[Environment]::SetEnvironmentVariable("LC_API_KEY", "your-api-key", "User")
```

These save the values as Windows user variables, so they never appear in code. Open a new PowerShell window, then check them. This shows only whether each value exists:

```
[bool]$env:LC_OID; [bool]$env:LC_API_KEY
```

Check: both lines print `True`.

Step 5: Download detections for the first time

Why: this proves the API key works before anything is built on it. `fetch_test.py` logs in, requests 3 days of detections, and saves 5 of them to `data\sample.json`.

```
cd "F:\Gitrepo\vigil mdr"
.\.venv\Scripts\Activate.ps1
python fetch_test.py
```

Check: it prints `saved data/sample.json`, and the file contains your detections.

Step 6: Convert detections into a simple format

Why: raw detections contain far more than the AI needs. The adapter in `adapters/limacharlie.py` keeps the host, user, time, process, parent and command line.

```
python convert_test.py
```

Problems found and fixed:

- ATT&CK techniques were empty, because they're stored in `detect_mtd.tags` and `rule_tags`.
- LSASS detections were blank, because their details sit inside an `EVENTS` list.
- A command line exposed an agent password, so a masking step now replaces secrets with `[REDACTED]`. Any exposed password must also be changed.

Check: converted detections show real values, not `unknown-host` or a 1970 date. Delete `data\sample.json` afterwards if it holds a real secret.

Step 7: Test the AI on one detection

Why: this confirms Ollama can read a real detection and return structured JSON. Install Ollama, then download the model:

```
ollama pull qwen3:4b
ollama list
pip install jsonschema
python step6_test.py
```

What these do: download the model, confirm it's installed, add the JSON checker, and send one LSASS detection to Ollama.

Check: the output shows `"ai_status": "ok"` and a verdict.

Step 8: Create the automatic job

Why: `poll.py` joins the pieces together so nobody has to run them by hand. Run it once first:

```
python poll.py
Get-Content data\poll.log -Tail 10
```

Check: the log shows a `fetched` line and no `run failed`. Then schedule it to run every 5 minutes with no window:

```
$dir = (Get-Location).Path
$action = New-ScheduledTaskAction -Execute "$dir\.venv\Scripts\pythonw.exe" -Argument
"`"$dir\poll.py`"" -WorkingDirectory $dir
$trigger = New-ScheduledTaskTrigger -Once -At (Get-Date).AddMinutes(1) -
RepetitionInterval (New-TimeSpan -Minutes 5) -RepetitionDuration (New-TimeSpan -Days
3650)
$settings = New-ScheduledTaskSettingsSet -StartWhenAvailable -AllowStartIfOnBatteries -
DontStopIfGoingOnBatteries -MultipleInstances IgnoreNew -ExecutionTimeLimit (New-
TimeSpan -Hours 1)
Register-ScheduledTask -TaskName 'Vigil Poller' -Action $action -Trigger $trigger -
Settings $settings
```

What this does: defines what to run, when to repeat it, and how to behave on battery or if a run is still going, then registers the task.

```
Get-ScheduledTaskInfo -TaskName 'Vigil Poller' | Select-Object LastRunTime,
LastTaskResult, NextRunTime
```

Check: `LastTaskResult` is `0`. While it's running, you may see `267009`.

Step 9: Group related alerts

Why: one attack can create many alerts. `correlate.py` groups them into one scored incident, so the AI and analyst see the whole story. Section 5 lists the rules. It runs automatically inside `poll.py`.

Check: the log shows an `incidents` count. In this project, 38 detections became 3 incidents.

Step 10: Improve the AI prompts

Why: better instructions give the model structure. `prompts/core.md` holds the main rules, and 11 playbooks cover areas such as execution and credential access. `build_prompt.py` chooses up to 4 by ATT&CK technique. `defang.py` makes indicators safe, and `validate.py` rejects invented event IDs.

```
python defang.py
python build_prompt.py path\to\incident.json
```

Check: the first shows masked values like `bad[.]com`. The second lists playbooks and the prompt size.

Step 11: Install PostgreSQL

Why: a database stores incidents reliably, and lets the portal search them. The official download returned a 403 error here, and drive C: had little space, so PostgreSQL came from conda-forge and was installed on F:.

```
$root = 'F:\PostgreSQL'
New-Item -ItemType Directory -Force $root | Out-Null
curl.exe -L -o "$root\micromamba.exe" 'https://github.com/mamba-org/micromamba-
releases/releases/latest/download/micromamba-win-64.exe'
$env:MAMBA_ROOT_PREFIX = "$root\mamba"
& "$root\micromamba.exe" create -y -p "$root\env" -c conda-forge --override-channels
'postgresql=17'
```

Next, create and start the database server. The password goes in a temporary file, which is deleted straight away:

```
$bin = "$root\env\Library\bin"
$pwFile = "$env:TEMP\pg-admin.txt"
$adminPassword = Read-Host 'Choose a postgres admin password'
[IO.File]::WriteAllText($pwFile, $adminPassword)
& "$bin\initdb.exe" -D "$root\data" -U postgres --pwfile=$pwFile -A scram-sha-256 -E
UTF8 --locale=C
Remove-Item -LiteralPath $pwFile
Add-Content -LiteralPath "$root\data\postgresql.conf" -Value "`nlisten_addresses =
'localhost'`nport = 5432" -Encoding ascii
& "$bin\pg_ctl.exe" start -D "$root\data" -l "$root\postgres.log"
& "$bin\pg_isready.exe" -h localhost -p 5432
```

Check: the last command prints `accepting connections`. `listen_addresses` keeps the database reachable from this laptop only.

Step 12: Start PostgreSQL at login

```
$cmd = "& '$bin\pg_ctl.exe' start -D '$root\data' -l '$root\postgres.log'"
$action = New-ScheduledTaskAction -Execute 'powershell.exe' -Argument "-NoProfile -
WindowStyle Hidden -Command `"$cmd`""
$trigger = New-ScheduledTaskTrigger -AtLogOn -User "$env:USERDOMAIN\$env:USERNAME"
Register-ScheduledTask -TaskName 'Vigil PostgreSQL' -Action $action -Trigger $trigger
```

Check: `Get-ScheduledTask -TaskName 'Vigil PostgreSQL'` shows the task. A real restart is still the final test.

Step 13: Create the database and tables

Why: the application uses its own limited login instead of the admin account.

```
$bytes = New-Object byte[] 24
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes)
$dbPassword = [BitConverter]::ToString($bytes).Replace('-', '')
[Environment]::SetEnvironmentVariable('VIGIL_DB_PASSWORD', $dbPassword, 'User')
$env:VIGIL_DB_PASSWORD = $dbPassword
$env:PGPASSWORD = Read-Host 'Enter the postgres admin password'
"CREATE ROLE vigil LOGIN PASSWORD '$dbPassword';`nCREATE DATABASE vigil OWNER vigil;" |
& "$bin\psql.exe" -h localhost -U postgres -d postgres -q -v ON_ERROR_STOP=1
Remove-Item Env:PGPASSWORD
pip install -r requirements.txt
python db.py
```

What this does: creates a random password, saves it, creates the login and database, then builds the tables from `schema.sql`.

Check: the last command prints `tables ready`. Run `python db.py` again after updating `schema.sql`.

Step 14: Build the web portal

Why: analysts need one place to review incidents. `portal.py` serves the pages in `templates/` and `static/`.

Set a password of at least 14 characters first. Nothing shows while you type:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\set-portal-password.ps1
```

Open a new PowerShell window, then start the portal, and leave the window open:

```
cd "F:\Gitrepo\vigil mdr"
.\.venv\Scripts\python.exe portal.py
```

Check: open `http://127.0.0.1:8080` and sign in. A bug found here: an early version of the password script added hidden characters, so correct passwords failed. The script now avoids that.

> Times from PostgreSQL first appeared in local time under a UTC label. The connection now forces UTC, because wrong timestamps can mislead an investigation.

Step 15: Add the analyst tools

These run inside the portal, so they need no extra commands:

- **Queue:** sorted by severity, then newest, with filters and a flag for failed AI analysis.
- **Incident page:** verdict card, attack chain, evidence links, indicators and a process tree from `process_tree.py`.
- **Review:** approve, edit, reject with a required reason, or close. Every decision goes in the audit trail.
- **Customer report:** available only after approval or editing, and leaves out raw events.
- **Dashboard:** charts from `charts.py`, built on the server with no chart library.

Check: open an incident, select an event ID, and confirm it jumps to the evidence.

Step 16: Publish the portal (optional)

> Add the login first. A public portal holds real hostnames, usernames, IP addresses and command lines. Skip this step unless you need it.

Start the portal in public mode:

```
$env:VIGIL_PUBLIC = '1'
.\.venv\Scripts\python.exe portal.py
```

Then, in a second window, start the free tunnel:

```
cloudflared tunnel --no-autoupdate --url http://127.0.0.1:8080
```

Check: open the `trycloudflare.com` address it prints. You should reach the login page, and signed-out visitors shouldn't see any incident data.

The address changes when the tunnel restarts, and the site stops if the laptop sleeps. To go offline at once:

```
Stop-Process -Name cloudflared
```

Step 17: Save the work to GitHub

Why: Git keeps a history of your code, and GitHub keeps a copy off your laptop. Create `.gitignore` first, so private data stays out:

```
.venv/
data/
__pycache__/
*.pyc
```

Then create a private repo and upload the code:

```
git init -b main
git add -A
git diff --cached --name-only
git commit -m "Add Vigil MDR detection pipeline"
gh repo create your-account/vigil-mdr --private --source . --remote origin --push
```

For later changes, use:

```
git add -A
git commit -m "Describe your change"
git push origin main
```

Check: review the list from `git diff --cached --name-only` before committing. Nothing from `data/` or `.venv/` should appear.

Step 18: Check that everything is healthy

```
Get-ScheduledTask -TaskName 'Vigil Poller'
Get-ScheduledTaskInfo -TaskName 'Vigil Poller' | Select-Object LastRunTime,
LastTaskResult, NextRunTime
Start-ScheduledTask -TaskName 'Vigil Poller'
Get-Content data\poll.log -Tail 5
.\.venv\Scripts\python.exe report.py
```

Check: the task is `Ready` or `Running`, the log has a recent `fetched` line, and the report command prints detection and incident counts.

13. Glossary

Term	Meaning
ATT&CK	MITRE's public catalog of attacker techniques, such as T1059.001 for PowerShell
API key	A password that lets a program, not a person, use a service
Atom	LimaCharlie's unique ID for a process
Correlation	Grouping related alerts into one incident
Defanging	Changing an indicator, such as `bad.com` to `bad[.]com`, so it can't be clicked
Detection	An alert raised when a rule matches an event
Environment variable	A named value stored by Windows that programs can read
IOC	Indicator of compromise, such as a suspicious IP address, domain or file hash
LSASS	The Windows process that handles logins, and a common target for credential theft
MDR	Managed Detection and Response, a service that monitors and investigates threats
Ollama	A tool that runs AI models on your own computer
Schema	A definition of the fields and format that data must follow

Sensor	Software on a machine that reports its activity
Sigma	A shared format for writing detection rules
Tunnel	A secure outbound link that gives a local site a public address
Virtual environment	A private folder of Python packages for one project